

Doubly Linked List – Traverse In Reverse And Space Complexity

Program :

```
void traverseInReverse()
{
    if (head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    Node *temp = head;
    while (temp->next != nullptr)
    { // Go to the last node
        temp = temp->next;
    }

    cout << "List (Reverse): NULL <- ";
    while (temp != nullptr)
    {
        cout << temp->data << " <-> ";
        temp = temp->prev;
    }
    cout << "NULL" << endl;
}
...
case 11:
    traverseInReverse();
    break;
```

Space Complexity

Auxiliary Space

Auxiliary space is the extra or temporary memory used by a function, excluding the space taken by the inputs themselves.

In the `traverseInReverse` function, the only new variable created is:

`Node *temp`: A single temporary pointer is used to find the end of the list and then traverse backward. The memory required for a single pointer is constant.

The function does not allocate any new nodes or data structures. Since the amount of extra memory used is fixed, the auxiliary space complexity is $O(1)$.

Auxiliary Space: $O(1)$

Input Space

Input space is the memory used to store the function's inputs. This can be analyzed from two perspectives.

1. Parameter – only Definition

This view considers only the space taken by the function's direct parameters. The function signature is `void traverseInReverse()`.

The function takes no parameters.

Under this definition, the input space is constant.

Input Space (Parameter-only): $O(1)$

2. Full – Data Definition

This broader view includes the space taken by the entire data structure the function operates on—the doubly linked list.

If the linked list contains n nodes, the total memory it occupies is proportional to n .

Under this definition, the input space is linear with respect to the size of the list.

Input Space (Full-Data): $O(n)$

Total Space Complexity

The total space complexity is the sum of the auxiliary space and the input space. The result depends on which definition of input space you use.

- **Common Interpretation (using parameter-only input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(1)}_{\substack{\downarrow \\ \text{Input}}} = O(1).$$

- **Holistic Interpretation (Full-data input):**

$$\text{Space} = \underbrace{O(1)}_{\substack{\downarrow \\ \text{Auxiliary}}} + \underbrace{O(n)}_{\substack{\downarrow \\ \text{Input}}} = O(n).$$

In most academic and interview contexts, the "space complexity" of a function refers to its auxiliary space. Therefore, the most common and expected answer for this function is $O(1)$. The two while loops that travel the list affect the function's time complexity ($O(n)$), but not its space complexity.

Algorithm: Traverse and Display List (Reverse)

This algorithm traverses a doubly linked list from end to beginning and prints the data of each node. The list starts at the node pointed to by START.

DREVERSETRAVERSE(START)

1. ***[CHECK IF LIST IS EMPTY]***

If START = NULL, then:

Write: "List is empty."

EXIT.

[End of If structure.]

2. ***[TRAVERSE TO THE LAST NODE OF THE LIST]***

a. Set PTR := START [PTR is a pointer used to traverse the list].

b. Repeat while NEXTLINK[PTR] ≠ NULL:

Set PTR := NEXTLINK[PTR] [Move forward to the next node].

[END of Loop]

3. ***[PRINT HEADER AND BEGIN REVERSE TRAVERSAL]***

a. Write: "List (Reverse): NULL < – "

b. Repeat while PTR ≠ NULL:

i. Write: INFO[PTR], " < –> " [Print the data of the current node].

ii. Set PTR := PREVLINK[PTR] [Move backward to the previous node].

[END of Loop]

4. ***[PRINT THE LIST TERMINATOR]***

Write: "NULL"

5. ***Exit***
-